FR-03 – Step 4: Boundary value analysis

Input: the classes in `FR-03_2_equivalence_classes.md` and the test cases in `FR-03_3_test_cases.md`.
Method: lecture S04, step 4.

Method

- BVA applies to **ordered** domains: lengths, counts, numeric values, time. For each ordered class, test **LB−1, LB, LB+1, UB−1, UB, UB+1** (lecture slide 26), plus the smallest/largest value the UI allows (slide 26, points 8 *and 9*).
- When a range has a single valid value (LB = UB), the six points collapse to three: LB−1, LB, UB+1.
- A boundary point that an existing step 3 test case already hits is **reused, not duplicated**. The "Test case" column says which one.
- Valid boundary points may be combined in one test case (as in step 3). Each invalid boundary point gets its own test case.

Variables with no ordered domain (no BVA)

Variable	Why no BVA
IN_01 Email	The spec gives a format, not a length or range.
IN_02 Back to login	A button: a set, not ordered.
IN_10 Confirm field in API	A set {present & matching, present & not matching, absent}.
IN_11 Step order	Two states; both covered in step 3 (TC_01, TC_42).
Special-character set `@$!%*?&`	A set of characters, not a range. Its count is ordered and is analysed below.

Boundary table

Default values when not under test: email `test@eshop.com`, OTP = X, new password `NewPass1!`, confirm = new password. V = expected accepted, I = expected rejected.

B1. OTP length (IN_03) – valid range [6, 6]

Point	Value	V / I	Test case
Smallest via UI (8*)	0 (empty)	I	FR-03_TC_15
LB−1	5 digits	I	FR-03_TC_16
LB = UB	6 digits (correct OTP)	V	FR-03_TC_01
UB+1	7 digits	I	FR-03_TC_17
Largest via UI (9*)	Type 20 digits; check whether the field cuts input at 6	Record	FR-03_TC_17 (note the field limit, if any)

B2. OTP value range in the format domain (IN_03) – `000000` … `999999`

Point	Value	V / I	Test case
Lowest 6-digit value	`000000` (not the issued OTP)	I	**FR-03_BVA_01**

Point	Value	V / I	Test case
Highest 6-digit value	`999999` (not the issued OTP)	I	**FR-03_BVA_02**

Edge values like `000000` catch code that treats an all-zero or "falsy" value as missing or as a wildcard.

B3. Generated OTP length (OUT_02) – output must always be exactly 6 digits

Point	Check	Test case
Output LB−1 / UB+1	Across 20 generated OTPs, none has 5 (or fewer) or 7 (or more) digits	**FR-03_BVA_03**
Output LB = UB	Every generated OTP has exactly 6 digits	**FR-03_BVA_03**

Output boundaries matter here: a random generator can produce too few digits (for example, by dropping leading zeros or using too small a range). Checking one OTP (TC_01) can miss this; checking many makes it visible.

B4. New password length (IN_04) – valid range [8, 128]

The lower bound is from FR-01. The upper bound 128 is a **tester decision** (EC_28: "> 128 is invalid").

Point	Value	Length	V / I	Test case
LB−1	`Pass12!`	7	I	FR-03_TC_25
LB	`Abcdef1!`	8	V	**FR-03_BVA_04**
LB+1	`Abcdefg1!`	9	V	**FR-03_BVA_05**
UB−1	`Aa1!` + 123 × `a`	127	V	**FR-03_BVA_06**
UB	`Aa1!` + 124 × `a`	128	V	**FR-03_BVA_07**
UB+1	`Aa1!` + 125 × `a`	129	I	FR-03_TC_26

B5. Character-class counts in the new password (IN_04) – each valid range [1, ∞)

FR-01 says "at least 1" uppercase, lowercase, digit and special character. Each count is ordered, with LB = 1.

Count	LB−1 = 0 (I)	LB = 1 (V)	LB+1 = 2 (V)
Uppercase	FR-03_TC_27	FR-03_BVA_04 (`Abcdef1!`)	**FR-03_BVA_08** (`ABcd12!@`)
Lowercase	FR-03_TC_28	**FR-03_BVA_09** (`ABCDEf1!`)	FR-03_BVA_08
Digit	FR-03_TC_29	FR-03_BVA_04	FR-03_BVA_08
Special from set	FR-03_TC_30	FR-03_BVA_04	FR-03_BVA_08

No upper bound is specified for any count.

B6. OTP age (IN_06) – valid range [0, T)

T is not in the spec. It is **measured at execution**: if TC_20 shows the OTP expires within 15 minutes, find T (from the error text, or by trying 1, 2, 5, 10 minutes). If the OTP never expires, BVA_10 and BVA_11 are "Not executed" and the missing expiry is reported as a SEC-07 bug.

Point	Value	V / I	Test case
LB	Submit immediately (age ≈ 0)	V	FR-03_TC_01
UB−1	T − 10 s	V	**FR-03_BVA_10**
UB	T	I (expired from T on)	**FR-03_BVA_11** (submit at T + 10 s)
UB+1	well past T (15 min)	I	FR-03_TC_20

The step is 10 seconds instead of 1 because a human tester cannot submit with 1-second precision. UB and UB+1 are therefore merged into "just after T" (BVA_11) and "well after T" (TC_20).

B7. OTP use count (IN_07) – valid range [0, 0]

Point	Value	V / I	Test case
LB = UB	0 (never used)	V	FR-03_TC_01
UB+1	1 (used in a successful reset)	I	FR-03_TC_21

B8. OTP request order (IN_09) – only the latest (n-th) OTP is valid

Point	Value	V / I	Test case
n = 1	Only OTP	V	FR-03_TC_01
n-th of n = 2	Latest	V	FR-03_TC_04
(n−1)-th of n = 2	Previous	I	FR-03_TC_23

B9. Confirm-password length relative to new password (IN_05)

The match rule implies an ordered check: a confirm value that is one character shorter or longer, with the same prefix, catches comparisons that only check the beginning of the string.

Point	Confirm (new password `NewPass1!`, 9 chars)	Length difference	V / I	Test case
−1	`NewPass1`	−1	I	**FR-03_BVA_12**
0	`NewPass1!`	0	V	FR-03_TC_01
+1	`NewPass1!!`	+1	I	**FR-03_BVA_13**

New BVA test cases

Precondition for all except BVA_03: P3 from step 3 (on step 2 with OTP X for `test@eshop.com`). Cleanup rule from step 3 applies: re-seed the database after any test case that changes the password.

TC ID	Test case name	Boundary	Precondition	Test steps (with data)	Expected result
FR-03_BVA_01	OTP at lowest 6-digit value	B2 low	P3 (X ≠ `000000`)	OTP `000000`, defaults for the rest, submit.	Incorrect-OTP error; password unchanged.
FR-03_BVA_02	OTP at highest 6-digit value	B2 high	P3 (X ≠ `999999`)	OTP `999999`, defaults for the rest, submit.	Incorrect-OTP error; password unchanged.

TC ID	Test case name	Boundary	Precondition	Test steps (with data)	Expected result
FR-03_BVA_03	Generated OTP is always 6 digits	B3	P1	Request an OTP for `test@eshop.com` 20 times (UI or `POST /api/forgot-password`). Record each OTP.	All 20 OTPs have exactly 6 digits.
FR-03_BVA_04	Password at minimum length, one of each required class	B4 LB; B5 LB (upper, digit, special)	P3	New password and confirm `Abcdef1!`, submit. Log in with it.	Success; login works.
FR-03_BVA_05	Password at minimum length + 1	B4 LB+1	P3	New password and confirm `Abcdefg1!`, submit.	Success.
FR-03_BVA_06	Password at maximum length − 1	B4 UB−1	P3	New password and confirm `Aa1!` + 123 × `a` (127 chars), submit. Log in with it.	Success; login works.
FR-03_BVA_07	Password at maximum length	B4 UB	P3	New password and confirm `Aa1!` + 124 × `a` (128 chars), submit. Log in with it.	Success; login works.
FR-03_BVA_08	Two of each required character class	B5 LB+1 (all four)	P3	New password and confirm `ABcd12!@`, submit.	Success.
FR-03_BVA_09	Exactly one lowercase letter	B5 LB (lowercase)	P3	New password and confirm `ABCDEf1!`, submit.	Success.
FR-03_BVA_10	OTP just before expiry	B6 UB−1	P3, T known	Wait until T − 10 s after getting X, submit with defaults.	Success.
FR-03_BVA_11	OTP just after expiry	B6 UB	P3, T known	Wait until T + 10 s after getting X, submit with defaults.	OTP-expired error; password unchanged.
FR-03_BVA_12	Confirm one character shorter	B9 −1	P3	New password `NewPass1!`, confirm `NewPass1`, submit.	Passwords-don't-match error.
FR-03_BVA_13	Confirm one character longer	B9 +1	P3	New password `NewPass1!`, confirm `NewPass1!!`, submit.	Passwords-don't-match error.

Summary

	Count
Ordered domains analysed	9 (B1–B9)
Boundary points identified	39
Points already hit by step 3 test cases (reused)	19
Points needing new test cases	20

	Count
New BVA test cases (several points share one valid test case)	13
Total FR-03 test cases (step 3 + step 4)	**42 + 13 = 55**

Notes for review

1. **B4's upper bound (128) is a tester decision**, not spec. Bugs from BVA_06, BVA_07 or TC_26 must say so.
2. **BVA_10 and BVA_11 depend on T,** which is measured at execution. If the OTP never expires, they become "Not executed" and the missing expiry is the bug.
3. **BVA_03 is an output boundary test.** It checks the system's own output, not user input, so it is easy for both people and AI to forget.
4. **B9 is an interpretation:** the spec only says the fields must match. Treating the length difference as an ordered domain is the tester's choice, made because prefix-only comparisons are a common bug.